



Relatório

Beatriz Santos - 50473
Inês Santos - 49436
Manoela Azevedo - 50034

Trabalho para a Unidade Curricular
Virtualização e Computação na Nuvem
1º ciclo de estudos em Informática Web, Móvel e na Nuvem

Docente: Mário Freire

Janeiro de 2024

Índice

Abstract.....	2
Introdução.....	3
Armazenamento na Nuvem.....	4
Firebase Cloud.....	5
Firebase Authentication.....	6
Firebase Realtime Database.....	6
Firebase Cloud Firestore.....	6
Firebase Cloud Functions.....	7
Firebase Cloud Storage.....	7
Firebase Hosting.....	7
Firebase Cloud Messaging (FCM).....	8
Firebase Performance Monitoring.....	8
Instalação da Firebase no Flutter.....	9
1. Instalação da CLI do Firebase e FlutterFire.....	9
2. Configurar o Flutter para utilização do Firebase.....	9
3. Inicializar o Firebase.....	9
4. Plug-ins do Firebase.....	9
Authentication e Firestore Cloud.....	11
Login1.....	11
ForgotPasswordPage.....	12
SignUp.....	13
Perfil.....	13
Conclusão.....	14
Bibliografia.....	15

Abstract

Neste Relatório, serão exploradas as principais etapas do projeto, desde a configuração inicial até a implementação de recursos específicos, utilizando os serviços oferecidos pelo Firebase. Será abordada detalhadamente a integração com o Firebase Cloud para armazenamento e gerenciamento de dados, bem como as funcionalidades de autenticação proporcionadas pelo Firebase Authentication. Além disso, discutir a utilização do Firebase Firestore para garantir uma persistência de dados eficiente e em tempo real por meio do Firestore.

Introdução

O relatório destaca o desenvolvimento de uma aplicação Flutter integrada no Firebase, uma plataforma de serviços em nuvem que oferece diversas funcionalidades essenciais para a construção de aplicações dinâmicas e escaláveis.

A fusão entre Flutter e Firebase tornou-se uma chave para a criação de soluções modernas e eficientes no ambiente em constante mudança do desenvolvimento de aplicações móveis e web. A ideia central deste projeto é que o Flutter, uma ferramenta de interface do utilizador versátil, e o Firebase, uma plataforma abrangente de serviços em nuvem fornecida pela Google, se integrem completamente.

O objetivo principal deste projeto é desenvolver e implementar uma aplicação Flutter que utilize a flexibilidade e as capacidades expressivas da interface Flutter e o poder dos serviços na nuvem do Firebase. O artigo explica detalhadamente o processo de integração, destacando o uso de serviços importantes do Firebase, como Autenticação, Firestore Cloud e outros.

Os princípios que fundamentam o armazenamento e a recuperação de dados numa nuvem são explicados, esclarecendo a base teórica do armazenamento na nuvem. Ao longo do processo de desenvolvimento, encontrar-se-á descrições detalhadas de cada passo, desde a configuração dos serviços do Firebase até à implementação da autenticação do utilizador e a sincronização de dados em tempo real.

Este projeto ilustra a colaboração entre Flutter e Firebase e mostra como eles podem criar uma aplicação robusta, escalável e adaptável. O relatório tem como objetivo fornecer informações úteis a programadores e entusiastas interessados em utilizar as capacidades do Flutter e Firebase nos próprios projetos devido à complexidade da jornada de desenvolvimento.

Armazenamento na Nuvem

O armazenamento na nuvem é um paradigma revolucionário que transformou a forma como lida-se com o armazenamento de dados. No passado, a gestão de dados exigia a aquisição e manutenção de servidores físicos, representando um desafio significativo para empresas e indivíduos que procuravam aproveitar os benefícios da computação num mundo cada vez mais digital.

No início da computação em nuvem, a necessidade de investir em servidores físicos era uma barreira para muitos que procuravam armazenar e aceder a dados de forma eficiente. A aquisição de servidores dedicados, sistemas de armazenamento e a configuração de uma infraestrutura eram processos demorados e dispendiosos. Além disso, a manutenção constante desses recursos exigia espaço físico, energia e uma equipa especializada.

A transição para o armazenamento na nuvem trouxe consigo uma revolução na forma como os dados são geridos. Este modelo baseia-se na utilização de servidores remotos virtuais, partilhados por utilizadores para armazenar e aceder a dados de forma flexível e eficiente. A ligação remota permite acesso de qualquer lugar, proporcionando mobilidade e flexibilidade aos utilizadores.

Ao contrário do paradigma tradicional, o armazenamento na nuvem elimina a necessidade de possuir e gerir servidores físicos. Os utilizadores podem armazenar, partilhar e aceder a dados sem a preocupação constante com a infraestrutura subjacente. Esta abordagem inovadora transferiu a responsabilidade pelo provisionamento, manutenção e gestão de servidores para os fornecedores de serviços na nuvem.

Assim como na computação sem servidor, a escalabilidade automática é uma característica essencial no armazenamento na nuvem. Os utilizadores podem ajustar dinamicamente o espaço de armazenamento conforme a procura, evitando desperdícios e otimizando os recursos disponíveis. A cobrança é baseada no uso real, proporcionando uma abordagem eficiente em termos de custos para diferentes padrões de utilização.

Mesmo com a presença de servidores, estes permanecem invisíveis para os utilizadores, eliminando a necessidade de gestão e configuração. Os fornecedores de serviços na nuvem assumem as operações de forma transparente, garantindo que os dados estejam prontos para serem acedidos quando necessário.

Em síntese, o armazenamento na nuvem oferece uma abordagem simplificada e eficaz para lidar com dados, proporcionando aos utilizadores a capacidade de armazenar e aceder a informações de maneira escalável e responsiva. Ao compreender e aplicar estes aspetos, empresas e indivíduos podem aproveitar ao máximo os benefícios desta revolução no gerenciamento de dados na era digital.

Firestore Cloud

A Firestore Cloud é um conjunto de serviços de nuvem criados pela Google que ajudam os desenvolvedores de aplicações móveis e da web a se desenvolver e a funcionar melhor. Uma ampla gama de ferramentas e recursos prontos para uso são fornecidos por essa plataforma, eliminando assim a necessidade dos desenvolvedores de construir e administrar infraestruturas complexas. A Firestore Cloud integra uma variedade de recursos, incluindo autenticação do utilizador, análise de aplicações e armazenamento de dados em tempo real. A palavra "Firestore Cloud" geralmente se refere aos serviços em nuvem que foram disponibilizados para o Firestore.

Firestore Authentication

Um desses serviços é a Firestore Authentication. A grande maioria das aplicações exige a identificação segura do utilizador, permitindo assim o armazenamento seguro de dados na nuvem e garantindo uma experiência de utilizador personalizada em todos os dispositivos usados. Ao permitir que os utilizadores autentiquem na sua app, o Firestore Authentication oferece bibliotecas de interface prontas para uso, SDKs fáceis de entender e serviços de back-end robustos.

Esta plataforma de autenticação suporta uma variedade de métodos, como autenticação por senha, números de telefone e integração com provedores federados de identidade populares, como Google, Facebook e Twitter, entre outros. As bibliotecas de interface pré-desenvolvidas e a sua implementação simples tornam o processo de autenticação muito mais fácil. Isso dá aos desenvolvedores as ferramentas necessárias para criar experiências de utilizador seguras e personalizadas nas aplicações.

Firestore Realtime Database

O Firestore Realtime Database é outra ferramenta de armazenamento em nuvem que permite que todos os clientes conectados se sincronizem em tempo real usando o formato JSON.

Os utilizadores compartilham uma instância centralizada da Realtime Database ao desenvolver aplicações que usam plataformas como Apple, Android e JavaScript SDKs. Uma experiência consistente e simultânea para todos os utilizadores é garantida por esse compartilhamento, que garante que as atualizações de dados sejam automaticamente distribuídas.

Firestore Cloud Firestore

Já o Cloud Firestore se destaca como um banco de dados adaptável que atende às necessidades de desenvolvimento de servidores, móveis e web. O Cloud Firestore, como o Firestore Realtime Database, permite que os ouvintes sincronizem os dados das aplicações clientes em tempo real. Além disso, permite a criação de aplicações responsivas que funcionam independentemente da latência da rede ou da conectividade com a Internet com suporte off-line para dispositivos móveis e web.

A flexibilidade do Cloud Firestore é notável, pois oferece uma estrutura de banco de dados que se adapta de forma dinâmica às necessidades de aplicações modernas. Uma capacidade avançada de consulta complementa essa flexibilidade, permitindo uma manipulação de dados mais eficaz e eficaz.

Além disso, destaca-se a excelente integração do Cloud Firestore com outros produtos tanto do Firebase quanto do Google Cloud, incluindo a sua colaboração eficaz com Cloud Functions. Isso aumenta ainda mais as oportunidades para o desenvolvimento de aplicações robustas e eficientes.

Firestore Cloud Functions

O Cloud Functions é uma estrutura sem servidor que permite que o código de back-end seja executado automaticamente em resposta a eventos causados por atividades em segundo plano, requisições HTTPS, chamadas do SDK Admin ou tarefas agendadas pelo Cloud Scheduler. O código escrito em JavaScript, TypeScript ou Python é armazenado na infraestrutura do Google Cloud e funciona em um ambiente gerenciado. Esta técnica facilita a implementação de lógica de servidor em aplicações Firebase e elimina a necessidade de lidar com escalabilidade e gestão de servidores.

Firestore Cloud Storage

O Cloud Storage para Firebase é uma solução de armazenamento baseada na infraestrutura ágil e segura do Google Cloud destinada a desenvolvedores de aplicações que precisam armazenar e distribuir conteúdo criado pelos utilizadores, como fotos e vídeos.

Este serviço, que é fácil de usar e eficiente, funciona como um sistema de armazenamento de objetos otimizado para escala que atende às necessidades do ecossistema do Google. Os SDKs do Firebase destinados ao armazenamento no cloud protegem o upload e o download de arquivos em aplicações Firebase com segurança integrada do Google, independentemente das condições de rede.

A variedade de tipos de conteúdo criados pelos utilizadores, como imagens, áudio e vídeo, podem ser armazenados nos SDKs do cliente. O SDK Admin do Firebase facilita a administração de buckets e URLs de download no servidor. Além disso, as APIs do Google Cloud Storage permitem acesso aos arquivos armazenados, facilitando a gestão de conteúdo eficiente e abrangente.

Firestore Hosting

O Firebase Hosting hospeda vários tipos de aplicações, como aplicações web, conteúdo estático e dinâmico, bem como microsserviços de forma rápida e segura. Representa uma plataforma de hospedagem de conteúdo da web de nível de produção que foi desenvolvida para atender às necessidades dos desenvolvedores.

O Firebase Hosting permite a implantação rápida de aplicações web com um comando, permitindo a entrega de conteúdo estático e dinâmico por meio de uma rede de distribuição de conteúdo (CDN) em todo o mundo. Além disso, é possível integrar o Firebase Hosting

com as funcionalidades da nuvem ou o Cloud Run. Isso permite que você crie e hospede microserviços diretamente na infraestrutura do Firebase. Uma abordagem abrangente para atender às demandas de desenvolvimento é fornecida por essa integração, o que permite a entrega eficiente de aplicações e serviços na nuvem.

Firestore Cloud Messaging (FCM)

O Firestore Cloud Messaging (FCM) facilita o envio confiável e sem custos associados de mensagens em várias plataformas.

Ao usar o FCM, é possível informar uma aplicação cliente sobre a disponibilidade de sincronização de dados, como novos e-mails. O envio de mensagens de notificação e as estratégias de reengajamento e retenção de utilizadores são facilitados por essa plataforma. É importante lembrar que uma mensagem pode transmitir uma carga útil de até 4.000 bytes para uma aplicação cliente, oferecendo flexibilidade na transmissão de informações em contextos específicos, como mensagens instantâneas.

Firestore Performance Monitoring

O Firestore Performance Monitoring é uma ferramenta projetada para fornecer informações detalhadas sobre o desempenho das suas aplicações em plataformas web, Android e Apple. Ao integrar o SDK de Monitoramento de Desempenho na sua aplicação, é possível coletar dados detalhados sobre o desempenho e, em seguida, revisar e analisar esses dados por meio do Console do Firestore. O que permite que você saiba onde o desempenho da sua app pode ser melhorado em tempo real.

Instalação da Firebase no Flutter

1. Instalação da CLI do Firebase e FlutterFire

Para conectar o Firebase ao Flutter foi necessário realizar a instalação da CLI do Firebase, um conjunto de ferramentas que permite gerir o projeto Firebase. O arquivo de instalação pode ser encontrado no website da Google Firebase. Foi utilizado o script de instalação automática, executando o comando cURL:

```
curl -sL https://firebase.tools
```

E fez-se o login à Firebase com a Conta Google através do comando:

```
firebase login
```

Para fazer a instalação da CLI do FlutterFire:

```
dart pub global activate flutterfire_cli
```

2. Configurar o Flutter para utilização do Firebase

Foi utilizado o seguinte código para a configuração no diretório do projeto Flutter

```
flutterfire configure
```

3. Inicializar o Firebase

No diretório do projeto Flutter, foi executado os seguintes códigos:

```
flutter pub add firebase_core  
flutterfire configure
```

No lib/main.dart, foi importado o plugin do Firebase:

```
import 'package:firebase_core/firebase_core.dart';  
import 'firebase_options.dart';
```

E inicializado no mesmo ficheiro com:

```
await Firebase.initializeApp(  
  options: DefaultFirebaseOptions.currentPlatform,  
);
```

Após essas alterações, foi necessário recriar o flutter:

```
flutter run
```

4. Plug-ins do Firebase

Usa-se o Firebase na aplicação Flutter para adicionar funcionalidades como armazenamento, autenticação e análise de dados. Cada recurso, como armazenar dados ou fazer login, tem a sua própria "ferramenta" no Firebase. Como o Flutter funciona em

iPhones, telefones Android e navegadores web, quando usa-se essas "ferramentas" na aplicação, elas funcionam automaticamente em todos esses lugares.

No diretório do projeto Flutter:

```
flutter pub add cloud_firestore  
flutter pub add firebase_core  
flutter pub add firebase_database  
flutter pub add firebase_auth  
flutter pub add cloud_firestore  
flutterfire configure
```

Foi recriado novamente o projeto Flutter:

```
flutter run
```

Authentication e Firestore Cloud

Um dos serviços da Firebase que utilizámos neste trabalho foi o de Authentication, este foi usado devido à necessidade de realizar o login na nossa aplicação. Logo o mesmo guardou as contas dos utilizadores (mail e password) e gerenciou-as conforme as operações que foi fazendo. Para além deste também utilizamos outros dos serviços da Firebase que foi o Firestore Cloud, este foi usado devido à necessidade de guardar mais informação sobre os utilizadores para além do email e password na nossa aplicação. Logo o mesmo guardou os dados dos utilizadores (nome, apelido, username e password).

Login1

O código da class Login1() cria a página onde os utilizadores realizaram o login na aplicação Flutter. Aqui estão as principais características:

Controladores de Texto

Como controladores de texto usamos emailController e passwordController, estes serviram para que pudéssemos guardar em variáveis os valores que os utilizadores preencheram nestes campos de entrada (TextField) de e-mail e senha.

Código final para a criação das variáveis:

```
final emailController = TextEditingController();  
final passwordController = TextEditingController();
```

Código final para a guardar os valores, e-mail e password:

```
TextField(  
  controller: emailController,  
  style: const TextStyle(  
    color: Color.fromRGBO(11, 8, 46, 1),  
    fontSize: 18,  
  ),  
  decoration: const InputDecoration(  
    hintText: 'Email'  
  ),  
) ,  
  
TextField(  
  controller: passwordController,  
  style: const TextStyle(  
    color: Color.fromRGBO(11, 8, 46, 1),  
    fontSize: 18,  
  ),  
  decoration: const InputDecoration(  
    hintText: 'Email'  
  ),  
) ,
```

Método signIn()

Este método é assíncrono e é chamado quando o botão "LOGIN" é pressionado. Utiliza o Firebase Authentication para realizar a autenticação do utilizador com e-mail e senha caso este já exista. Se a autenticação for bem-sucedida, ele navega para a página Perfil().

Código final para o Firebase Authentication:

```
Future signIn(String email, String password, BuildContext context) async {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email,
    password: password,
  );
  Navigator.push(
    context, MaterialPageRoute(builder: (context) => Perfil())
  );
}
```

Código final para o Botão "LOGIN":

```
ElevatedButton(
  onPressed: () async {
    await signIn(
      emailController.text.trim(),
      passwordController.text.trim(),
      context
    );
  },
  style: ElevatedButton.styleFrom(
    backgroundColor: Color.fromRGBO(13, 10, 48, 1),
    shadowColor: Colors.transparent,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(10),
    ),
  ),
  child: const Text("LOGIN",
    style: TextStyle(
      fontFamily: "Brandon",
      fontSize: 26,
      color: Color.fromRGBO(255, 255, 255, 1),
    ),
  ),
),
```

ForgotPasswordPage

O código da class ForgotPasswordPage() cria a página onde o utilizador poderá escrever o seu email e pedir para registar uma nova password por motivos de esquecimento, com o auxílio do Authentication podemos realizar esta operação com um só comando.

Código final para envio de e-mail:

```

MaterialButton(
  onPressed: () async{
    try {
      await FirebaseAuth.instance.sendPasswordResetEmail(
        email: emailController.text.trim(),
      );
      showDialog(
        context: context,
        builder: (context){
          return AlertDialog(
            content: Text(
              'Password reset link sent!\n Check your email.'
            ),
          );
        },
      );
    } on FirebaseAuthException catch (e) {
      print(e);
      showDialog(
        context: context,
        builder: (context){
          return AlertDialog(
            content: Text(e.toString()),
          );
        },
      );
    }
  },
)
}

```

SignUp

O código da class SignUp() cria a página onde o utilizador poderá registar-se caso não tenha conta. Aqui estão as principais características:

Controladores de Texto

Como controladores de texto usamos emailController, nomeController, apelidoController, usernameController e passwordController, estes servirão para que pudéssemos guardar em variáveis os valores que os utilizadores preencheram nestes campos de entrada (TextField) de e-mail, nome, apelido, username e password.

Código final para a criação das variáveis:

Exemplo do código final para a guardar os valores, e-mail:

Método signIn()

Este método é assíncrono e é chamado quando o botão "LOGIN" é pressionado. Utiliza o Firebase Authentication para realizar a autenticação do utilizador com e-mail e senha caso este já exista. Se a autenticação for bem-sucedida, ele navega para a página Perfil().

Código final para o Firebase Authentication:

Código final para o Botão "LOGIN":

Perfil

O código Perfil representa a página principal após o login bem-sucedido. Aqui estão as principais características:

Botão de Logout

Exibe um ícone de seta para a esquerda, que, quando pressionado, realiza o logout e retorna à página inicial (MyHomePage). Código final:

Informações do Utilizador

Exibe o e-mail do utilizador autenticado. Código final:

Conclusão

Neste projeto, exploramos como desenvolver aplicações Flutter e aproveitar as capacidades do Firebase de forma eficiente. O nosso objetivo principal era criar uma aplicação sólida, rápida e escalável, proporcionando uma ótima experiência ao utilizador e também aproveitando os recursos avançados da nuvem.

O Firebase Cloud foi utilizado para armazenar e recuperar dados de maneira eficaz, garantindo respostas rápidas e confiáveis. Além disso, a Firebase Authentication adicionou uma camada de segurança importante, assegurando a autenticidade e autorização adequada dos utilizadores.

Um ponto importante foi também a implementação do Firebase Firestore através do Firestone. Isso não apenas forneceu uma solução eficiente para o banco de dados em tempo real, como também possibilitou a sincronização contínua de dados entre dispositivos, criando uma experiência fluída e consistente.

Em suma, este projeto ressalta a colaboração eficaz entre Flutter e Firebase no desenvolvimento de softwares modernos.

Bibliografia

<https://firebase.google.com/docs/flutter/setup?platform=ios&hl=pt>

https://www.youtube.com/watch?v=hy0NtR0NW4Q&list=PLlvRDpXh1Se4wZWOWs8yapI8AS_fwDHzf